HEART STROKE PREDICTION

USING MACHINE LEARNING

By:
SONU RAJENDRAN

HEART STROKE PREDICTION
USING MACHINE LEARNING

A Project Report
Submitted in partial fulfillment of the
Requirements for the award of the Degree of

MASTER OF SCIENCE(COMPUTER SCIENCE)

By

Sonu Rajendran(SMSC2425148)

Under the esteemed guidance of

Dr.Manoj Singh

SIES College of Arts, Science & Commerce
RISE WITH EDUCATION
NAAC REACCREDITED - 'A' GRADE
ISO 9001 : 2008

(Autonomous)

DEPARTMENT OF COMPUTER SCIENCE

SIES COLLEGE OF ARTS,SCIENCE AND COMMERCE(AUTONOMOUS)

SION(W),MUMBAI,400022

MAHARASHTRA

2022

Head of Department

Prof.Manoj Singh

Project Guide

Prof.Manoj Singh

Submitted by

Sonu Rajendran

SMS2425148

SIES College of Arts, Science & Commerce

RISE WITH EDUCATION

NAAC REACCREDITED - 'A' GRADE

ISO 9001 : 2008

S.I.E.S College of Arts, Science and Commerce Sion(W), Mumbai – 400 022.

CERTIFICATE

This is to certify that the project entitled, "**HEART STROKE PREDICTION – USING MACHINE LERNING**", is bonafied work of **SONU RAJENDRAN** bearing Seat.No: **SMSC2425148** submitted in partial fulfillment of the requiremnets for the award of degree of **MASTER OF SCIENCE** in **COMPUTER SCIENCE** from **UNIVERSITY OF MUMBAI.**

Prof. In-Charge

Dr. Manoj Singh

External Examiner

Head of the Department

Prof. Manoj Singh

College Seal

Date

DECLERATION

We hereby declare that the project entitled, “Heart Stroke Prediction Using Machine Learning Algorithm” done at SIES COLLEGE OF ARTS, SCIENCE & COMMERCE(AUTONOMOUS)- SION(W), has not been in any case duplicated to submit to any other university for the reward of any degree. To the best of our knowledge other than us, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the reward of degree of MASTER OF SCIENCE (COMPUTER SCIENCE) to be submitted as 3rd semester project as part of our curriculim.

Name and Signature of the Student’s Involved:

ABSTRACT

In the medical field, the diagnosis of heart disease is the most difficult task. The diagnosis of heart disease is difficult as a decision relied on grouping of large clinical and pathologicaldata. Due to this complication, the interest increased in a significant amount between the researchers and clinical professionals about the efficient and accurate heart disease prediction. In case of heart disease, the correct diagnosis in early stage is important as timeis the very important factor. Heart disease is the principal source of deaths widespread, and the prediction of Heart Disease is significant at an untimely phase. Machine learning in recent years has been the evolving, reliable and supporting tools in medical domain and has provided the greatest support for predicting disease with correct case of training and testing. The main idea behind this work is to study diverse prediction models for the heart disease and selectingimportant heart disease feature using Random Forests algorithm. Random Forests is theSupervised Machine Learning algorithm which has the high accuracy compared to other Supervised Machine Learning algorithms such as logistic regression, KNN, SVM and Decision Tree. By using RandomForests algorithm we are going to predict if a person is likely to get a heart stroke or not.

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

Cardiovascular diseases (CVDs) are one of the leading causes of mortality globally, accounting for 32 percent of all deaths and affecting 17.9 million people worldwide. Among these, the two most significant CVDs are coronary heart attacks and strokes, which together account for 85 percent of all cases. A heart attack occurs when oxygen or blood flow to the heart muscle is interrupted, while a stroke is caused by a blockage in the artery that supplies blood to the brain. Although these diseases are serious, the risk factors that contribute to them are very similar. These factors include an unhealthy diet, smoking, diabetes, a sedentary lifestyle, excessive alcohol consumption, high blood pressure, and family history. Detecting a heart attack and seeking medical attention as soon as possible not only increases the chances of survival but also helps prevent future heart issues.

Machine learning has emerged as one of the most challenging fields in modern technology. It is a form of artificial intelligence where models can analyze data, identify patterns, and make predictions with minimal human intervention. Various machine learning techniques can be applied to predict heart stroke in individuals. Data mining techniques involve extracting valuable and hidden information from large datasets, and machine learning, a subfield of data mining, effectively handles well-formatted large-scale datasets. In the medical field, machine learning can be used for analysis, detection, and prediction of various diseases.

The primary aim of this project is to provide doctors with a tool to predict heart strokes as early as possible, helping to administer effective treatment and avoid severe consequences. Machine learning plays a crucial role in identifying hidden patterns in data, which can then be analyzed to predict heart strokes. Based on input characteristics obtained from the dataset used to train the model, the proposed system predicts heart strokes in individuals using a variety of machine learning methods, such as Random Forests and K-Nearest Neighbors.

Therefore, in this paper, a machine learning algorithm is proposed for theimplementation of a heart disease prediction system which was validated on two open accessheart disease prediction datasets. Data mining is the computer based process of extractinguseful information from enormous sets of databases. Data mining is most helpful in anexplorative analysis because of nontrivial information from large volumes of evidence.Medical data mining has great potential for exploring the cryptic patterns in the data sets ofthe clinical domain.

These patterns can be utilized for healthcare diagnosis. However, the available rawmedical data are widely distributed, voluminous and heterogeneous in nature .This data needsto be collected in an organized form.

Data mining technology affords an efficient approach to the latest and indefinite patternsin the data. The information which is identified can be used by the healthcare administratorsto get better services. Heart disease was the most crucial reason for victims in the countrieslike India, United States. In this project we are predicting the heart disease usingclassification algorithms. Machine learning techniques like Classification algorithms suchas Random forest, Logistic Regression, KNN, SVM and Decision Tree are used to explore different kinds of heart based problems.

Chapter 2

LITERATURE REVIEW

Heart disease kills many people, and people who smoke are more likely to develop heart disease. If the ECG (electrocardiogram pattern) is a ventricular echo, the person is at risk for a heart attack. Many previous studies have used machine learning-based methods to predict heart attacks. Chen et al. Heart attack prediction guide. They used data from 82 stroke patients for analysis using two artificial neural network (ANN) models to predict accuracy and achieved 79% and 95% accuracy, respectively. Zhong et al. Cardiology studies have been conducted to predict mortality in stroke patients. A total of 15,099 patients were included in this study for cardiac diagnosis. They use deep neural network technology to analyze heartbeat. The authors used PCA to extract medical history and predict heart attacks. Govindarajan et al. collected data from 507 patients to conduct a study combining text extraction and machine learning to classify heart disease. Their study used various machine learning (ML) algorithms for ANN training, with the SGD algorithm achieving the highest result of 95%. Amini et al. A study found that there were determinants of arterial performance in 807 healthy and diseased participants, 50 of whom had stroke, diabetes, heart disease, smoking tobacco, hyperlipidemia, and alcohol consumption. Excludes dangerous situations. They use the c4.5 decision tree algorithm (95% accuracy) and the nearest neighbor method (94% accuracy). Shin et al. conducted research on AI-assisted heart rate prediction. Their study used the Cardiovascular Health Study (CHS) dataset to predict stroke. For main content analysis, features were extracted using the decision tree method. They used a distributed neural network approach to build the model. The model achieved 97% accuracy. Chin et al. A study shows that premature beats can be heard. The main goal of their research is to develop a strategy that can be used to detect large beats using CNNs. They collected 256 images to train and test the CNN model. They used the data representation method to improve the images captured in the imaging plan by pre-planning to remove the unwanted, non-existent stroke area. The CNN method achieves up to 90% accuracy. After reviewing previous papers, the main idea behind the proposed scheme is to establish a stroke prediction based on communication. Based on the accuracy, precision, regression and f-value, we studied the KNN, decision tree and random forest classification algorithms and selected the best classification method to predict heart disease. Murat et al. It has become a common practice to provide measurement information and increase the deep understanding of patterns in the classification of ECG data. In this study, they used electrocardiogram data from 5 tissues (a total of 100,022 beats from the MIT-BIH Arrhythmia Dataset) to evaluate the most comprehensive study. Hong et al. The review authors evaluated deep learning methods for electrocardiogram data through modeling or post-generation thinking. The author used ECG (DL) deep learning models published by Google Scholar, PubMed, Digital Appendices & Library Mission from January 1, 2010 to February 29, 2020. Various deep learning methods proposed by Ebrahimi et al. are reviewed. Most of the analyses used at least some form of deep learning to extract and classify features. Determination of popular theory and prediction (CNN) with thirteen deep layers of fully convolutional neuron ensembles by Acharya et al. This method is time consuming, does not allow the use of real objects, and can control for performance differences among learners. As in ANN, the overall performance of the final version of CNN will be determined by the weighted image of the neighborhood and the location of the previous layer. The pooling method reduces the size of the output neurons of the convolution process, reduces computational time, and prevents overfitting. The accuracy, specificity, and sensitivity of the proposed method were 88.67%, 90.00%, and 95.00%, respectively. Acharya et al. In fact. It was determined that the baseline blurring and measurement of electrocardiograms were successful but further research is needed. The accuracy value is 86.67% and the specificity is 93.75%. The design has proven to be good in various performance measurements but can be modified for real use. According to Limaye and Deshmukh, the input of the ECG consists of low-frequency alarms between 0.5 and 100 Hz. The device that allows filtering of ECG recordings is called a cardiac video display unit. A low pass filter (LPF) is used to filter out non-noise. Mbachu et al. Models of LPF, HPF and BSF provided by Kaiser Windows, where three filters are connected in series, giving a signal in the range of 0 to 100 Hz, giving a signal at 13 dB every 200 voids, with negative reporting of the cut-off. Deiss et al. used a Hamming window to create a virtual design with 50 Hz interference, resulting in 13.4 dB attenuation. Lijan et al. reviewed more than eighty cardiac MRI, computed tomography, and single-photon computed tomography studies, including imaging in vascular coagulation and echocardiography. Introduced the basics of state-of-the-art deep search algorithms. Kaplan Belkaya et al. published a review of electrocardiographic signatures, analyzing 1,538 recordings, including heart rate measurements, pulse rate, heart rate monitoring, emotional intelligence, and biometric identification. They also record the best performance of the first ECG alarm system. First of all, it is important to learn its benefits as a different approach.

Cheng et al. published a report on the evaluation of heart failure. In their analysis using data from 82 stroke patients, two ANN models were used to find the accuracy of 79% and 95% respectively. Conducted a study to predict the mortality of stroke patients. In their study, they used 15,099 patients to analyze the incidence of heart disease. They use deep neural network approach to diagnose heart disease. The authors used PCA to extract clinical history data and predict cardiovascular disease. The area under the curve (AUC) value is 83%. Cene et al. conducted a study on automatic detection of heart disease at early stage. The main goal of their research is to develop a system for the first heart treatment using CNNs. They collected 256 images and tested the CNN model.

Chapter 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM:

Clinical decisions are often made based on doctors' intuition and experience rather than on the knowledge rich data hidden in the database. This practice leads to unwanted biases, errors and excessive medical costs which affects the quality of service provided to patients. There are many ways that a medical mis diagnosis can present itself. Whether adoctor is at fault, or hospital staff, a misdiagnosis of a serious illness can have very extremeand harmful effects. The National Patient Safety Foundation cites that 42% of medical patients feel they have had experienced a medical error or missed diagnosis. Patient safety issometimes negligently given the back seat for other concerns, such as the cost of medicaltests, drugs, and operations. Medical Misdiagnoses are a serious risk to our healthcare profession. If they continue, then people will fear going to the hospital for treatment. We can put an end to medical misdiagnosis by informing the public and filing claims and suits againstthe medical practitioners at fault.

Disadvantages:

• Prediction is not possible at early stages.
• In the Existing system, practical use of collected data is time consuming.
• Any faults occurred by the doctor or hospital staff in predicting would lead to fatalincidents.
• Highly expensive and laborious process needs to be performed before treating the patientto find out if he/she has any chances to get heart disease in future.

3.2 PROPOSED SYSTEM:

This section depicts the overview of the proposed system and illustrates all of thecomponents, techniques and tools are used for developing the entire system. To develop anintelligent and user-friendly heart disease prediction system, an efficient software tool isneeded in order to train huge datasets and compare multiple machine learning algorithms.After choosing the robust algorithm with best accuracy and performance measures, it will be implemented on the development of the web-based application for detecting and predicting heart disease risk level.

3.3 ALGORITHMS:

3.3.1 Logisitic Regression:

A popular statistical technique to predict binomial outcomes (y = 0 or 1) is LogisticRegression. Logistic regression predicts categorical outcomes (binomial / multinomial valuesof y). The predictions of Logistic Regression (henceforth, LogR in this article) are in the formof probabilities of an event occurring, i.e. the probability of y=1, given certain values of inputvariables x. Thus, the results of LogR range between 0-1.LogR models the data points using the standard logistic function, which is an S-shaped curve also called as sigmoid curve and is given by the equation:

Logistic Regression Assumptions:

• Logistic regression requires the dependent variable to be binary.
• For a binary regression, the factor level 1 of the dependent variable should represent thedesired outcome.
• Only the meaningful variables should be included.
• The independent variables should be independent of each other·
• Logistic regression requires quite large sample sizes.
• Even though, logistic (**logit**) regression is frequently used for binary variables (2 classes), itcan be used for categorical dependent variables with more than 2 classes.
• In this case it's called Multinomial Logistic Regression.

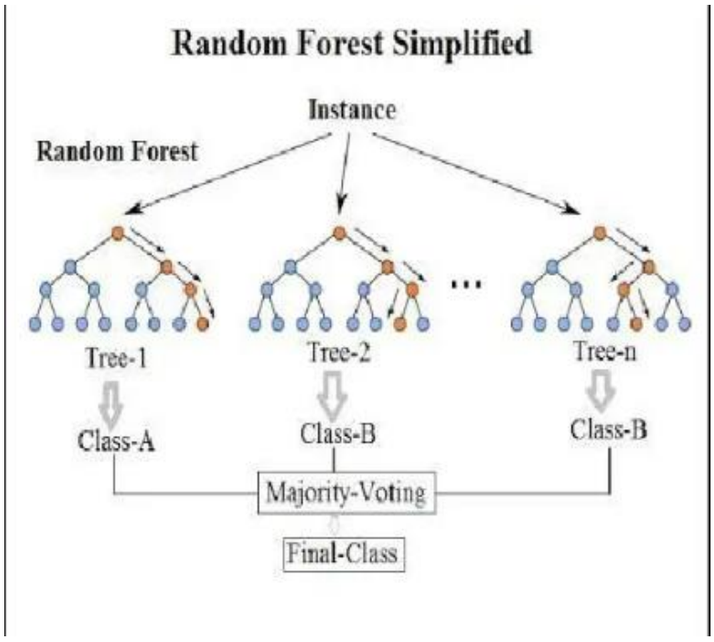
3.3.2 Random Forest:

Random forest is a supervised learning algorithm which is used for both classification as wellas regression .But however ,it is mainly used for classification problems. As we know that aforest is made up of trees and more trees means more robust forest .Similarly ,random forest creates decision trees on data samples and then gets the prediction from each of them and finally selects the best solution by means of voting .It isensemble method which is better than a single decision tree because it reduces the over-fitting by averaging the result.

Working of Random Forest with the help of following steps:

• First ,start with the selection of random samples from a given dataset.
• Next ,this algorithm will construct a decision tree for every sample .Then it will get the prediction result from every decision tree .
• In this step, voting will be performed for every predicted result.
• At last ,select the most voted prediction results as the final prediction result.

The following diagram will illustrates its working:



K.3.3 **KNN:**

K-Nearest Neighbors (KNN) is a simple, instance-based, and non-parametric supervised learning algorithm used for classification and regression tasks. It assumes that similar data points are close to each other in feature space. KNN is a lazy learning algorithm that doesn't explicitly learn a model during training. Instead, it stores the entire training dataset and makes predictions by looking at the k closest points (neighbors) in the feature space during testing. A new point is classified by the majority label of these neighbors.

The proximity between data points is measured using distance metrics (e.g., Euclidean distance), and the classification is made based on the most common label among the nearest neighbors. It's effective in scenarios where similar data points tend to have the same class label.

How KNN Works:

- **Training Phase**: KNN does not perform explicit training. It simply stores the training data.
- **Prediction Phase**:
  - For a given test data point, the algorithm calculates the distance between the test point and all training points using a metric (e.g., Euclidean, Manhattan, or Minkowski distance).
  - It identifies the **K nearest neighbors** (where K is a hyperparameter).
  - For classification: The test point is assigned to the majority class among its neighbors.
  - For regression: The output is the mean (or weighted average) of the neighbors' values.

Advantages:
- Simple to implement and understand.
- No assumptions about data distribution.

Disadvantages
- Computationally expensive for large datasets (distance calculation for all points).
- Sensitive to irrelevant features or unscaled data.

3.3.4 SVM:

Support Vector Machine (SVM) is a robust supervised learning algorithm used for classification and regression. It aims to find the optimal hyperplane that best separates data points of different classes in a high-dimensional space.

SVM works by finding the hyperplane (in 2D, this is a line; in higher dimensions, it's a plane) that best separates the classes with the largest margin. The margin is the distance between the hyperplane and the closest data points (called support vectors) from each class.

SVM focuses on maximizing the margin, as a larger margin is thought to lead to better generalization. For data that isnt linearly separable, SVM uses kernel functions (like the RBF or polynomial kernel) to transform the data into a higher-dimensional space where a linear separator can be found.

How SVM Works

- **Linear SVM**:
  - Finds a hyperplane (decision boundary) that maximizes the margin between two classes.
  - Support vectors are the closest points to the hyperplane.
- **Non-linear SVM**:
  - For non-linearly separable data, SVM uses **kernel functions** (e.g., RBF, polynomial) to transform the data into higher-dimensional space where a linear separation is possible.

Advantages

- Effective for high-dimensional spaces.
- Works well for both linear and non-linear decision boundaries.
- Resistant to overfitting, especially with proper regularization.

Disadvantages

- Computationally expensive for large datasets.
- Performance is sensitive to the choice of kernel and hyperparameter.

3.3.5 Decsion Tree:

A Decision Tree is a flowchart-like supervised learning algorithm used for classification and regression. It splits data into branches based on feature thresholds, leading to decision nodes and leaves representing class labels or output values.

A Decision Tree splits the data recursively based on the value of a chosen feature that best separates the classes (using metrics like Gini impurity or information gain). At each node, the data is divided into branches based on this feature, and the process continues until the leaves contain homogeneous data points (i.e., belonging to a single class).

The tree is built top-down by selecting the best feature at each step that minimizes classification error. The splitting continues until certain stopping criteria are met (e.g., maximum depth, minimum samples in a node). Decision Trees can easily overfit, but they are interpretable and provide a clear path of decision-making based on the features.

How Decision Tree Works

- **Splitting Criteria**:
  - For classification: Uses metrics like **Gini Impurity** or **Information Gain** to choose the best feature and threshold for splitting.
  - For regression: Uses **Mean Squared Error** (MSE) or **Variance Reduction**.
- **Tree Construction**:
  - Starts with the root node containing all data.
  - Iteratively splits data into branches to minimize impurity or error.
- **Prediction**:
  - A test point follows the tree's branches until it reaches a leaf node, which determines the predicted value or class.

Advantages

- Easy to visualize and interpret.
- Captures non-linear relationships without requiring feature scaling.

Disadvantages

- Prone to overfitting, though this can be mitigated with pruning or setting depth limits.
- Sensitive to small variations in the data.

3.4 FEASIBILITY STUDY:

A Feasibility Study is a preliminary study undertaken before the real work of a projectstarts to ascertain the likely hood of the projects success. It is an analysis of possiblealternative solutions to a problem and a recommendation on the best alternative.

3.4.1 Economic Feasibility:

It is defined as the process of assessing the benefits and costs associated with thedevelopment of project. A proposed system, which is both operationally and technically. feasible, must be a good investment for the organization. With the proposed system the usersare greatly benefited as the users can be able to detect the fake news from the real news andare aware of most real and most fake news published in the recent years. This proposedsystem does not need any additional software and high system configuration. Hence the proposed system is economically feasible.

3.4.2 Technical Feasibilty:

The technical feasibility infers whether the proposed system can be developedconsidering the technical issues like availability of the necessary technology, technicalcapacity, adequate response and extensibility. The project is decided to build using Python.Jupyter Note Book is designed for use in distributed environment of the internet and for
the professional programmer it is easy to learn and use effectively. As the developingorganization has all the resources available to build the system therefore the proposed systemis technically feasible.

3.4.3 Operational Feasibilty:

Operational feasibility is defined as the process of assessing the degree to which a proposed system solves business problems or takes advantage of business opportunities. Thesystem is self-explanatory and doesn’t need any extra sophisticated training. The system
has built-in methods and classes which are required to produce the result. The application can behandled very easily with a novice user. The overall time that a user needs to get trained is 14less than one hour. As the software that is used for developing this application is veryeconomical and is readily available in the market. Therefore the proposed system isoperationally feasible.

Chapter 4

SOFTWARE REQUIREMENTS SPECIFICATION

4.1 INTRODUCTION TO REQUIREMENT SPECIFICATION

Software Engineering by James F Peters & WitoldPedrycz Head First Java by Ka. A Software Requirements Specification (SRS) is a description of particular software product, program or set of programs that performs a set of functions in a target environment (IEEE Std. 830-1993).

a. Purpose

The purpose of software requirements specification specifies the intentions and intended audience of the SRS.

b. Scope

The scope of the SRS identifies the software product to be produced, the capabilities, application, relevant objects etc. We are proposed to implement Passive Aggressive Algorithm which takes the test and trained data set from the

c. Definitions, Acronyms and Abbreviations Software Requirements Specification

It's a description of a particular software product, program or set of programs that performs a set of function in target environment.

d. References

IEEE Std. 830-1993, IEEE Recommended Practice for Software Requirements Specifications thy Sierra and Bert Bates.

e. Overview

The SRS contains the details of process, DFD's, functions of the product, user characteristics. The non-functional requirements if any are also specified.

f. Overall description

The main functions associated with the product are described in this section of SRS. The characteristics of a user of this product are indicated. The assumptions in this section result from interaction with the project stakeholders.

4.2 REQUIREMENT ANALYSIS

Software Requirement Specification (SRS) is the starting point of the software developing activity. As system grew more complex it became evident that the goal of the
entire system cannot be easily comprehended. Hence the need for the requirement phase
arose. The software project is initiated by the client needs. The SRS is the means of translating
the ideas of the minds of clients (the input) into a formal document (the output of the requirement phase.) Under requirement specification, the focus is on specifying what

has
been found giving analysis such as representation, specification languages and tools, and
checking the specifications are addressed during this activity. The Requirement phase
terminates with the production of the validate SRS document.

Producing the SRS document is the basic goal of this phase. The purpose of the
Software Requirement Specification is to reduce the communication gap between the clients
and the developers. Software Requirement Specification is the medium though which the
client and user needs are accurately specified. It forms the basis of software development. A
good SRS should satisfy all the parties involved in the system.

4.2.1 Product Perspective:

The application is developed in such a way that any future enhancement can be easily
implementable. The project is developed in such a way that it requires minimal maintenance.
The software used are open source and easy to install. The application developed should be
easy to install and use. This is an independent application which can be easily run on to any
system which has Python installed and Jupiter Notebook.

4.2.2 Product Features:

The application is developed in a way that ‘Heart disease’ accuracy is predicted using
Random Forest. The dataset is taken from https://www.datacamp.com/community/tutorials/scikit-learn-credit-card. We can compare
the accuracy for the implemented algorithms. User characteristics Application is developed
in such a way that its users are v Easy to use v Error free 20 v Minimal training or no training
v Patient regular monitor Assumption & Dependencies It is considered that the dataset taken
fulfils all the requirements.

4.2.3 Domain Requirements:

This document is the only one that describes the requirements of the system. It is meant for
the use by the developers, and will also be the bases for validating the final Heart disease
system. Any changes made to the requirements in the future will have to go through a formal
change approval process. User Requirements User can decide on the prediction accuracy to
decide on which algorithm can be used in real-time predictions. Non Functional
Requirements ÿ Dataset collected should be in the CSV format ÿ .The column values should

be numerical values ÿ Training set and test set are stored as CSV files ÿ Error rates can be
calculated for prediction algorithms product.

4.2.4 Requirements Efficiency:

Less time for predicting the Heart Disease Reliability: Maturity, fault tolerance and

recoverability. Portability: can the software easily be transferred to another environment,
including install ability.

4.2.5 Usability:

How easy it is to understand, learn and operate the software system Organizational
Requirements: Do not block the some available ports through the windows firewall. Internet
connection should be available Implementation Requirements The dataset collection, internet
connection to install related libraries. Engineering Standard Requirements User Interfaces
User interface is developed in python, which gets input such stock symbol.

4.2.6 Hardware Interfaces:

Ethernet on the AS/400 supports TCP/IP, Advanced Peer-to-Peer Networking (APPN)
and advanced program-to-program communications (APPC). ISDN To connect AS/400 to an
Integrated Services Digital Network (ISDN) for faster, more accurate data transmission. An
ISDN is a public or private digital communications network that can support data, fax, image,
and other services over the same physical interface. We can use other protocols on ISDN,
such as IDLC and X.25. Software Interfaces Anaconda Navigator and Jupiter Notebook are
used.

4.2.7 Operational Requirements:

a) Economic: The developed product is economic as it is not required any hardware interface
etc.
Environmental Statements of fact and assumptions that define the expectations of the system
in terms of mission objectives, environment, constraints, and measures of effectiveness and
suitability (MOE/MOS). The customers are those that perform the eight primary functions of
systems engineering, with special emphasis on the operator as the key customer.
b) Health and Safety: The software may be safety-critical. If so, there are issues associated
with its integrity level. The software may not be safety-critical although it forms

part of a
safety-critical system.

- For example, software may simply log transactions. If a system must be of a high integrity level and if the software is shown to be of that integrity level, then the hardwar must be at least of the same integrity level.
- There is little point in producing 'perfect' code in some language if hardware and syste software (in widest sense) are not reliable. If a computer system is to run software of a high integrity level then that system should not at the same time accommodate softwarof a lower integrity level.
- Systems with different requirements for safety levels must be separated. Otherwise, the highest level of integrity required must be applied to all systems in the same environment.

4.3 SYSTEM REQUIREMENTS

4.3.1 Hardware Requirements

Processor : above 500 MHz
Ram : 4 GB
Hard Disk : 4 GB
Input device : Standard Keyboard and Mouse.
Output device : VGA and High Resolution Monitor.

4.3.2 Software Requirements

Operating System : Windows 7 or higher
Programming : Python 3.6 and related libraries
Software : Anaconda Navigator and Jupyter Notebook.

4.4 SOFTWARE DESCRIPTION

4.4.1 Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace. It provides constructs that enable clear programming on both small and large scales. Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library. Python interpreters are available for many operating systems. C Python, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of its variant implementations. C Python is managed by the non-profit Python Software Foundation.

4.4.2 Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. The name Pandas is derived

from the word Panel Data – an Econometrics from Multidimensional data.
In 2008, developer Wes McKinney started developing pandas when in need of high
performance, flexible tool for analysis of data. Prior to Pandas, Python was majorly used for
data mining and preparation. It had very little contribution towards data analysis. Pandas
solved this problem.
Using Pandas, we can accomplish five typical steps in the processing and analysis of data,
regardless of the origin of data — load, prepare, manipulate, model, and analyze.
Python with Pandas is used in a wide range of fields including academic and commercial domains
including finance, economics, Statistics, analytics, etc.
Key Features of Pandas:

- Fast and efficient Data Frame object with default and customized indexing.
- Tools for loading data into in-memory data objects from different file formats.
- Data alignment and integrated handling of missing data.
- Reshaping and pivoting of date sets.
- Label-based slicing, indexing and sub setting of large data sets.
- Columns from a data structure can be deleted or inserted.
- Group by data for aggregation and transformations.
- High performance merging and joining of data.
- Time Series functionality.

4.4.3 NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays. It is the
fundamental package for scientific computing with Python.
It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities 24
- Besides its obvious scientific uses, NumPy can also be used as an efficient multi-
  dimensional container of generic data. Arbitrary data-types can be defined using Numpy
  which allows NumPy to seamlessly and speedily integrate with a wide variety of
  databases.

4.4.4 Sckit-Learn

- Simple and efficient tools for data mining and data analysis\
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

4.4.5 Matploit lib

- Matplotlib is a python library used to create 2D graphs and plots by using python

Scripts.

- It has a module named pyplot which makes things easy for plotting by providing
  feature to control line styles, font properties, formatting axes etc.
- It supports a very wide variety of graphs and plots namely - histogram, bar charts,
  power spectra, error charts etc.

4.4.6 Jupyter Notebook

- The Jupyter Notebook is an incredibly powerful tool for interactively developing and
  presenting data science projects.
- A notebook integrates code and its output into a single document that combines
  visualizations, narrative text, mathematical equations, and other rich media.
- The Jupyter Notebook is an open-source web application that allows you to create
  and share documents that contain live code, equations, visualizations and narrative
  text.

- Uses include: data cleaning and transformation, numerical simulation, statistical
  modeling, data visualization, machine learning, and much more.
- The Notebook has support for over 40 programming languages, including Python, R,
  Julia, and Scala.
- Notebooks can be shared with others using email, Drop box, Git Hub and the Jupyter
  Notebook.
- Your code can produce rich, interactive output: HTML, images, videos, LATEX, and
  custom MIME types.
- Leverage big data tools, such as Apache Spark, from Python, R and Scala. Explore
  that same data with pandas, scikit-learn, ggplot2, Tensor Flow.

Chapter 5

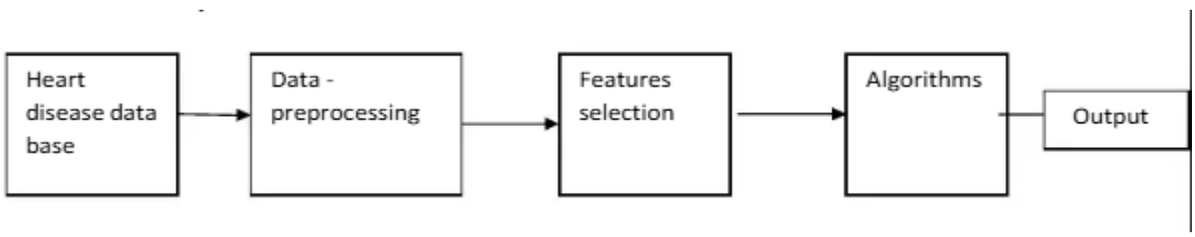
SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The below figure shows the process flow diagram or proposed work. First we collected the

Cleveland Heart Disease Database from UCI website then pre-processed the dataset and

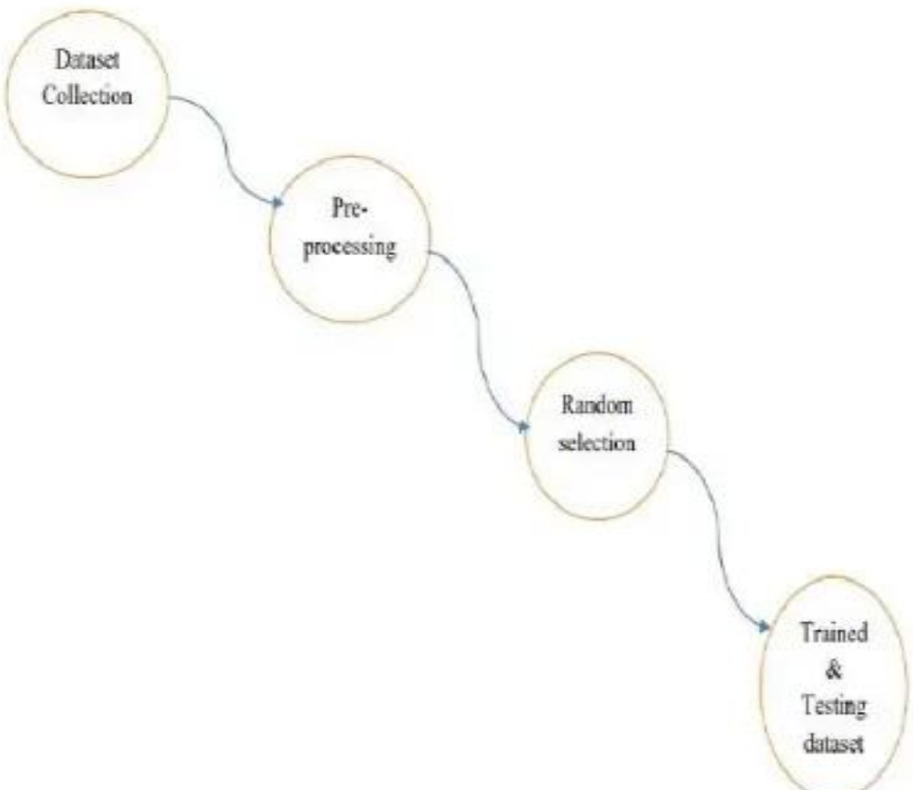
select 16 important features.



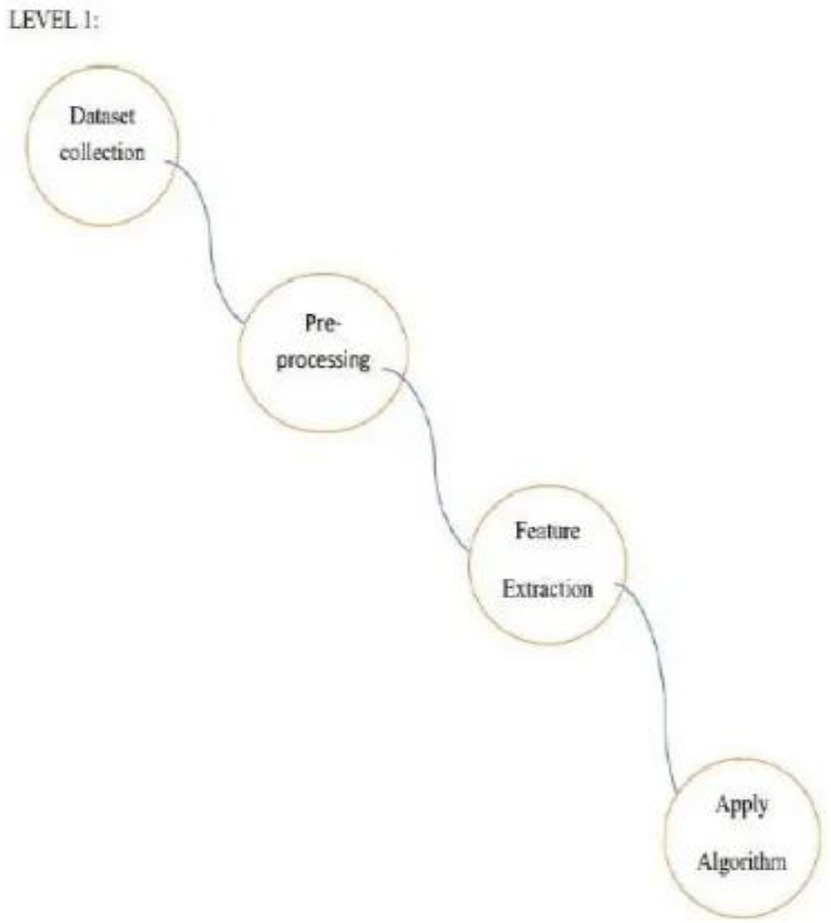
5.2 DATA FLOW DIAGRAM

The data flow diagram (DFD) is one of the most important tools used by system analysis.Data flow diagrams are made up of number of symbols, which represents system components.Most data flow modeling methods use four kinds of symbols: Processes, Data stores, Dataflows and external entities.These symbols are used to represent four kinds of system components. Circles in DFDrepresent processes. Data Flow represented by a thin line in the DFD and each data store hasa unique name and square or rectangle represents external entities.

Level 0:



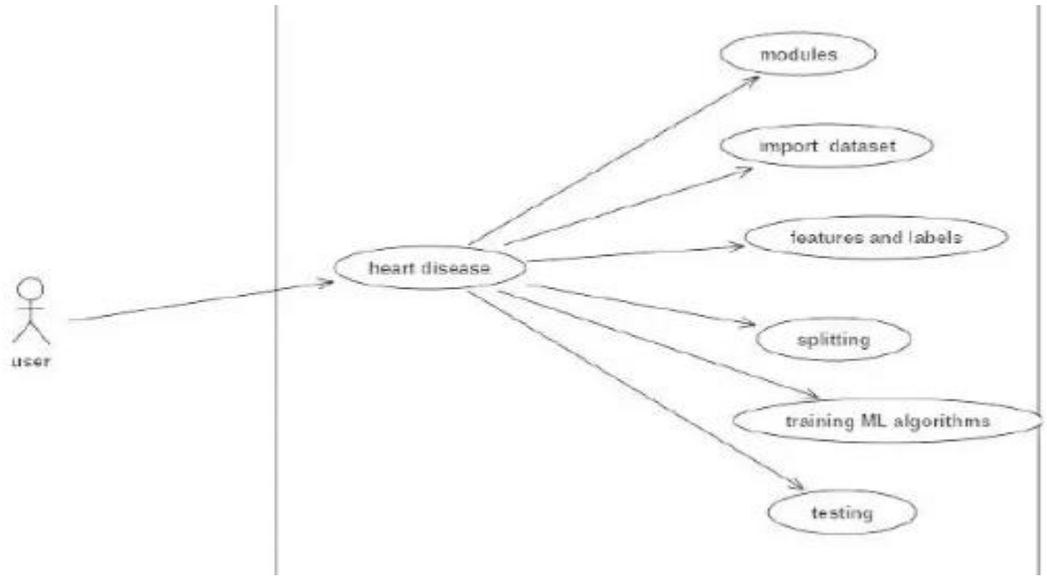
Level 1:



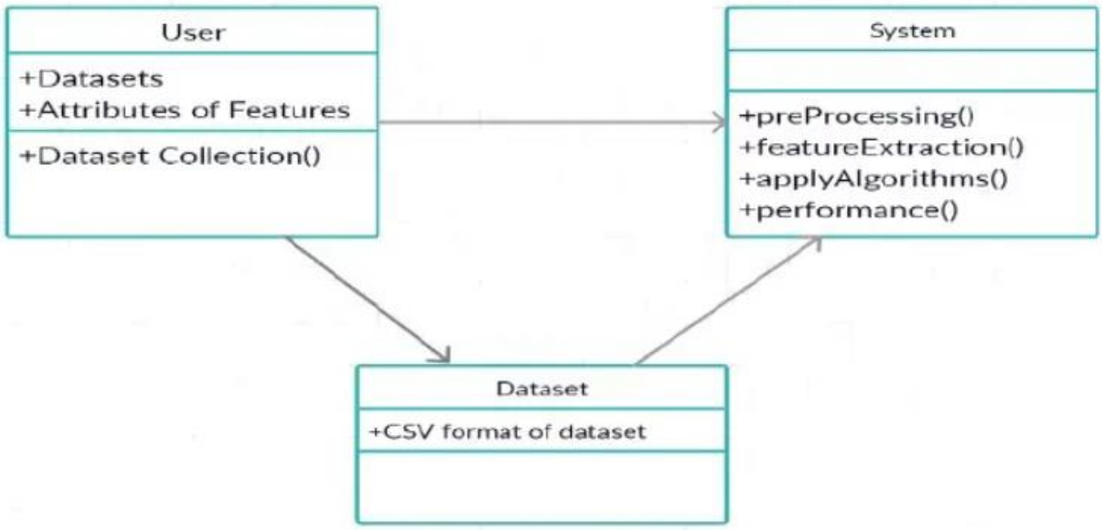
5.3 UML DIAGRAMS

5.3.1 USE-CASE DIAGRAM:

A use case diagram is a diagram that shows a set of use cases and actors and theirrelationships. A use case diagram is just a special kind of diagram and shares the samecommon properties as do all other diagrams, i.e a name and graphical contents.



5.3.2 CLASS DIAGRAM:


User
+Datasets
+Attributes of Features
+Dataset Collection()
System
+preProcessing()
+featureExtraction()
+applyAlgorithms()
+performance()
Dataset
+CSV format of dataset

Chapter 6
IMPLEMENTATION

6.1 Steps:

- ***Import Data*:** Importing the dataset for data pre-processing which is in csv format. The dataset consists of 12 parameters and that parameters are id, age, gender, hypertension, worktype, residence type, heart disease, avg level of glucose, body mass index (bmi), marital status, smoking status, and stroke. Importing the required libraries i.e. pandas, Numpy, Matplotlib, Seaborn, and warnings libraries. Import the CSV data as pandas Dataframe. Explore the shape, size, describe functions. Statsitical Inference.

- ***Data Mining:***

Exploratory data analysis: The dataset contains string values which I had to integer encoding for string values. Like for example, the string 'Male' 'Female' with '0' and '1' and so on for other categorial features as the model will only procees the data in binary language.

a. id : This attribute means person's id. It's numerical data.
b. Age : This attribute means a person's age. It's numerical data.
c. Gender : This attribute means a person's gender. It's categorical data.
d. Hypertension : This attribute means that this person is hypertensive or not. It's numerical data.
e. work type : This attribute represents the person work scenario. It's categorical data.
f. residence type : This attribute represents the person living scenario. It's categorical data.
g. heart disease : This attribute means whether this person has a heart disease person or not. It's numerical data.
h. avg glucose level : This attribute means what was the level of a person's glucose condition. It's numerical data.
i. Bmi : This attribute means body mass index of a person. It's numerical data.
j. ever married : This attribute represents a person's married status. It's categorical data.
k. smoking Status : This attribute means a person's smoking condition. It's categorical data.
l. Stroke : This attribute means a person previously had a stroke or not.

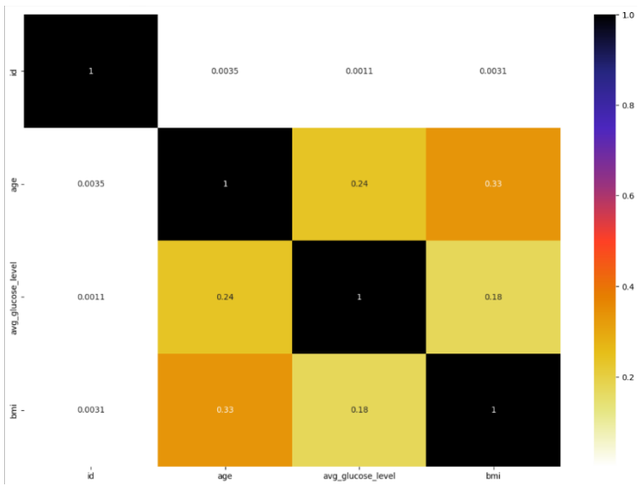
Sl.No	Attribute Name	Type
1	Gender	Numerical type
2	Age	Category type
3	Hypertension is there or not	Category type
4	Heart disease stroke	Numerical type
5	Marital status	Category type
6	Work type	Category type
7	Residence type	
8	Average_glucoselevel	Numerical type
9	Bmi (body mass index)	Numerical type
10	Smoking status	Category type
11	Stroke	Numerical type

Univariate Analysis: The term univariate analysis refers to the analysis of one variable prefix “uni” means “one.” The purpose of univariate analysis is to understand the distribution of values for a single variable. It is commonly used for both categorical and numerical data.

- age, bmi,avg_glucose_level are the only continuous features.
- id is a primary key which is of no importance.
- hypertension, heart_disease, stroke are Categorical features but they are encoded.

Multivariate Analysis: It is the analysis of more than one variable.

- Check Multicolinearity in Numerical Features:



- **Check Multicolinearity in Categorial Features:**

A chi-squared test (also chi-square or $\chi 2$ test) is a statistical hypothesis test that is valid to perform when the test statistic is chi-squared distributed under the null hypothesis, specifically Pearson’s chi-squared test. A chi-square statistic is one way to show a relationship between two categorical variables.

Null Hypothesis (): The Feature is independent of target column (No-Correlation).

Alternative Hypothesis (): The Feature and Target column are not independent (Correlated).

Col	Hypothesis	Result
0	gender	Fail to Reject Null Hypothesis
1	ever_married	Reject Null Hypothesis
2	work_type	Reject Null Hypothesis
3	Residence_type	Fail to Reject Null Hypothesis
4	smoking_status	Reject Null Hypothesis
5	hypertension	Reject Null Hypothesis
6	heart_disease	Reject Null Hypothesis
7	stroke	Reject Null Hypothesis

Gender and Residence_type are independent of the target columnn.

- **Fill Null Values:** we should always handle the missimg values in the datset while always preparing the data The dataset had 201 rows of missing values out of the total 5110 rows in the dataset which was of the feature ‘bmi’. Filled missing values using ‘mean’ with the average count, 201 rows of null values filled with the average mean count that is 28.89.

Initial Analysis Report-

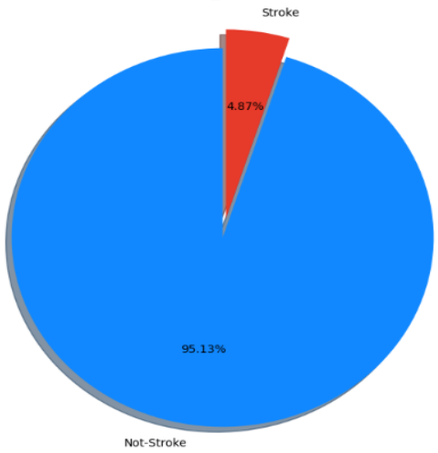
- There were missing values in the bmi.
- The ‘id’ col can be deleted as each row has unique values.
- Resident_type & Gender cols are not correlated with stroke.

- The stroke column is the target to predict.

- ***Visualization:***

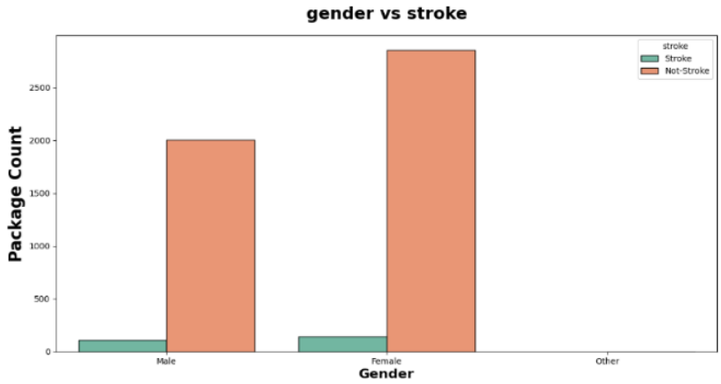
Visualising each & every feature of the dataset.

- Visualize the target fetaure-



From the chart it is clear that the Target variable is imbalanced i.e. here number of not-stroke has more count .

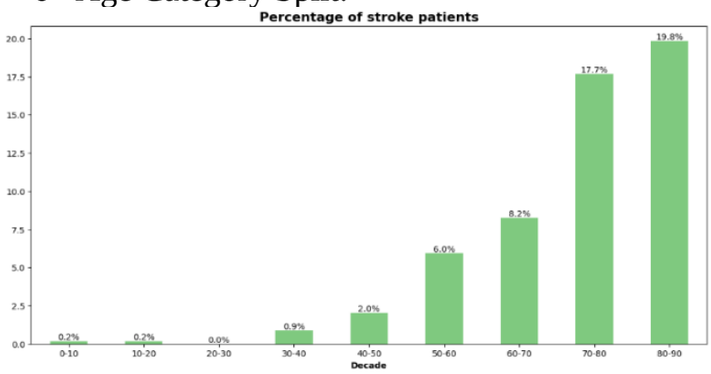
Do Men or Women have more chance of stroke?



Insights:

- As per the chart there is no much difference between the stroke of male and female.
- This Feature has no impact on the Target Variable.

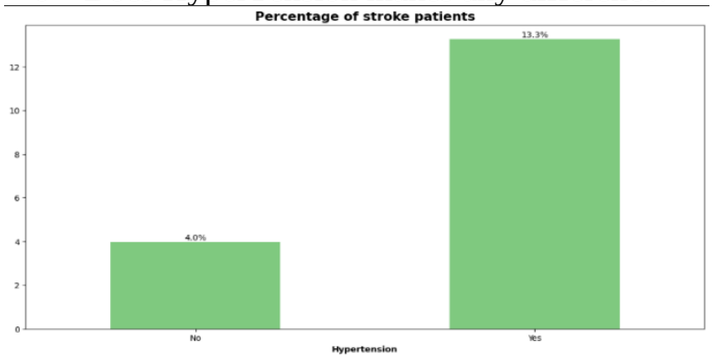
- Age Category Split:



Insights:

- As per the Chart Senior age Group has the more chance of heart stroke followed by adults.
- Children and teen have least chance of the stroke because of high physical activity. As the age increases there is higher chance of heart stroke.

- Does HyperTension makes any difference:

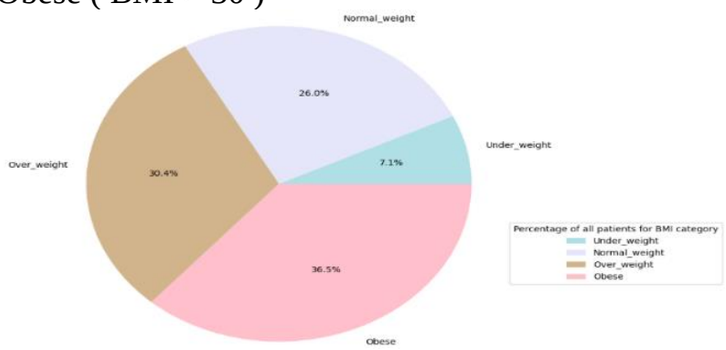


Insights:

- Even though the patients with no hypertension and having chance of stroke are high but remember the data is imbalanced
- In comparision with the number of plople in both categories patients with hypertension have more chance of stroke.

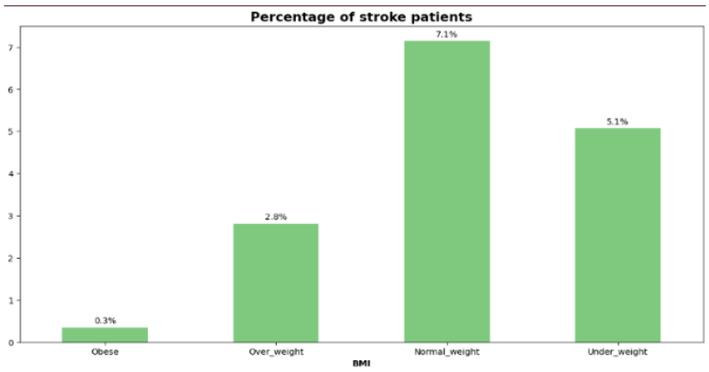
- Impact of BMI:

BMI can be divided into four categories:
Under_weight (BMI < 18.5)
Normal_weight (18.5 < BMI < 25)
Over_weight (25 < BMI < 30)
Obese (BMI > 30)



- We have most number of obese patients followed by the

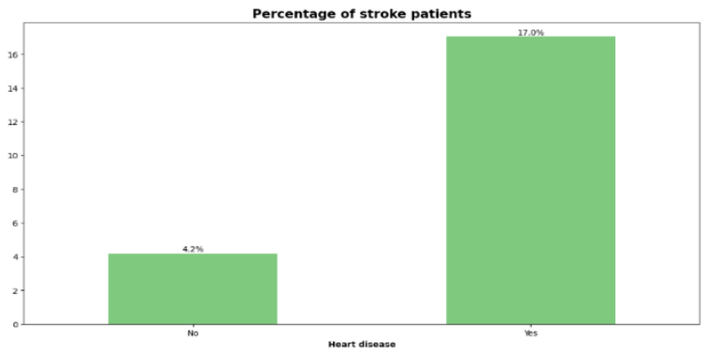
Over_weight totally comprises of nearly 65% of the total patients and have least number of patients with underweight.



Insights:

- Obese and overweight people have more chance of stroke.

- Heart Diseases vs Stroke:



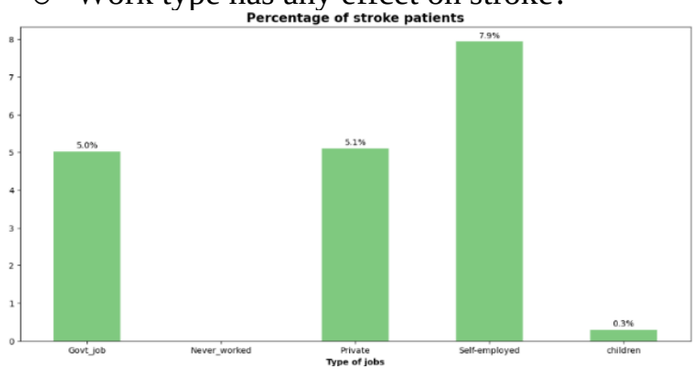
- Patients with heart disease have high chance of heart attack the the healty people.

- Does marriage has any affect on stroke:

Insights:

- Surprisingly married people have more chance of stroke which doesn't make any sense.
- It may because the married peple are aged than the unmarried and age is factor playing major role.

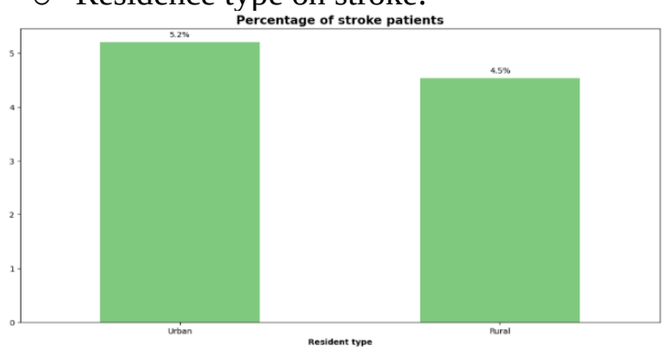
- Work type has any effcet on stroke?



Insights:

- Self-employed patients have high chance of heart stroke.
- Government and provate job employes have nearly same chance of heart stroke.
- Unemployed and children have least or near zero chance of stroke.

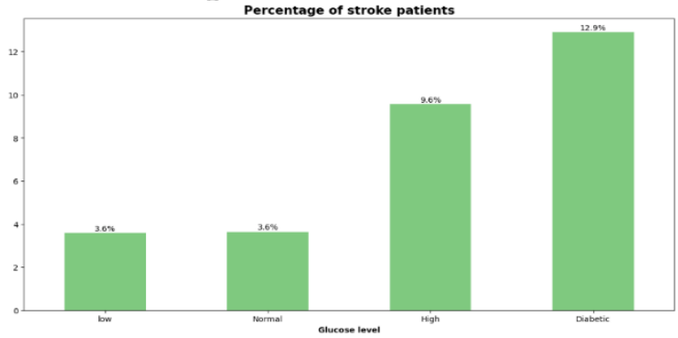
- Residence type on stroke:



Insights:

- Urban residents silghtly edge over the rural residents here.

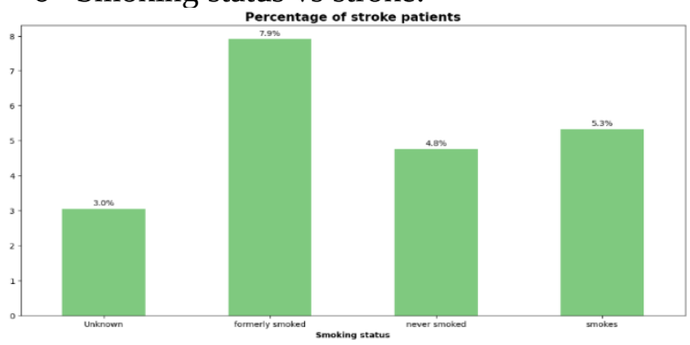
- Effect of glucose level on stroke:



Insights:

- We can see that the patients with low or normal glucose levels have less chance of stroke.
- High chance of stroke is found in diabetic patients followed by high glucose level.

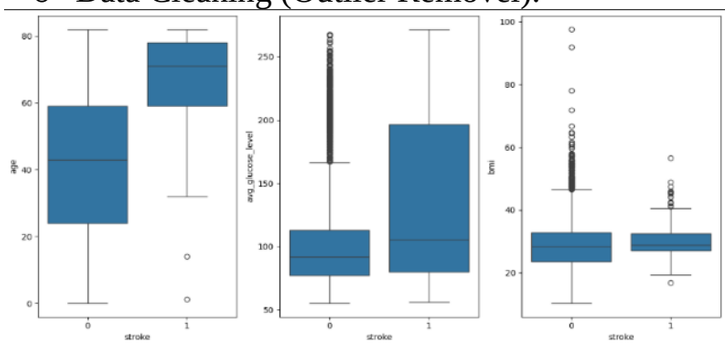
- Smoking status vs stroke:



Insights:

- Formerly_smoked seems to have high chance of stroke
- Smokes and never smoked have nearly same chance
- Smoking have a long term impact and may not be seen at the beginning stages.

- *Data Pre-Processing:*
- Data Cleaning (Outlier Removel):



Insights:

-The stroke column is the target to predict.

-There are outliers in the avg_glucose_level, bmi columns.

-Gender and Residence_type has less or no effect on target column.

- We will keep the outliers in the glucose_level and bmi since they are reasons to decide on whether the person has a stroke or not.

- **Feature Engeneering:**

Types of fetaures:

- Categorial Fetaures: These are features that are data in predefined categories.

There values are typically names or labels, not numeric values. Since ML models cannot handle non-numeric data, these features need to be converted to numeric values.

Num of Categorical Features : 5
['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']

- Numeric Features: These represent data that is quantitative and can be measured. These features include both discrete and continuous variables.

Num of Numerical Features : 6
['age', 'hypertension', 'heart_disease', 'avg_glucose_level', 'bmi', 'stroke']

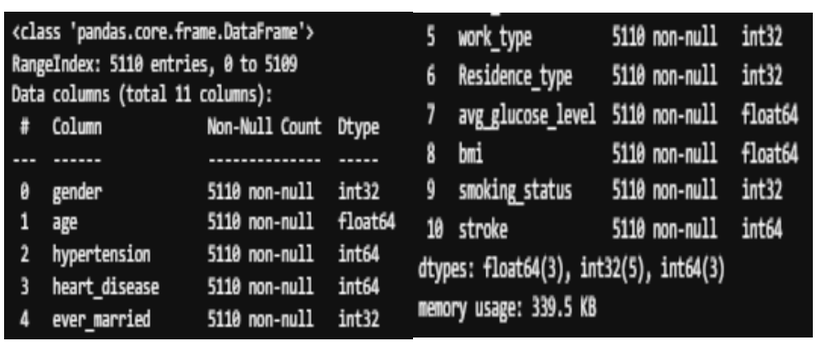
- Discrete Features: Discrete features are numerical features that take only specific, distinct values, typically integers.

Num of Discrete Features : 3
['hypertension', 'heart_disease', 'stroke']

- Continuous Features: Continuous features are numerical features that can take any value within a given range.

Num of Continuous Features : 3
['age', 'avg_glucose_level', 'bmi']

- **Label Encoding (Normalization):** Assigning numerical values to the categorial features.



After label encoding each of the 10 fetaures are either in int values or float numbers.

- **Data Partitioning**: It is the process of dividing the available dataset into different subsets to train, validate, and test a machine learning model. It ensures that the model is evaluated on data it hasn't seen before and generalizes well to new, unseen data.
- Splitting the data for train and test
- x means the fetaures and Y means the target variable, 80% of data would be used for training and the 20% would be used for testing
- X ---X_train, X_test
- Y ---Y_train, Y_test

- **Normalization:** We need to scale down the data, in the dataset the mean between age and hypertension the distance was too large and hence the model will not be able to travel that much distance so it needed to be scaled down. The mean was scaled to zero and std to 1 so the data will be in a range.

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status
count	5110.000000	5110.000000	5110.000000	5110.000000	5110.000000	5110.000000	5110.000000	5110.000000	5110.000000	5110.000000
mean	0.414286	43.226614	0.097456	0.054012	0.656164	2.167710	0.508023	106.147677	28.893237	1.376908
std	0.493044	22.612647	0.296607	0.226063	0.475034	1.090293	0.499985	45.283560	7.698018	1.071534
min	0.000000	0.080000	0.000000	0.000000	0.000000	0.000000	0.000000	55.120000	10.300000	0.000000
25%	0.000000	25.000000	0.000000	0.000000	0.000000	2.000000	0.000000	77.245000	23.800000	0.000000
50%	0.000000	45.000000	0.000000	0.000000	1.000000	2.000000	1.000000	91.885000	28.400000	2.000000
75%	1.000000	61.000000	0.000000	0.000000	1.000000	3.000000	1.000000	114.090000	32.800000	2.000000
max	2.000000	82.000000	1.000000	1.000000	1.000000	4.000000	1.000000	271.740000	97.600000	3.000000

- ***Model Training – Model selection:***

- The Machine learning models selected for this project were the Decision Tree, Logistic Regression, KNN, SVM and the Random Forest algorithms.
- 80% of data were used for the training phase and 20% of data were used for the testing phase.
- All of this 5 algorithms were trained and tested using the X_train, Y_train, X_test and Y_test. Y_pred was used to predict the results of X_test for each of the models.

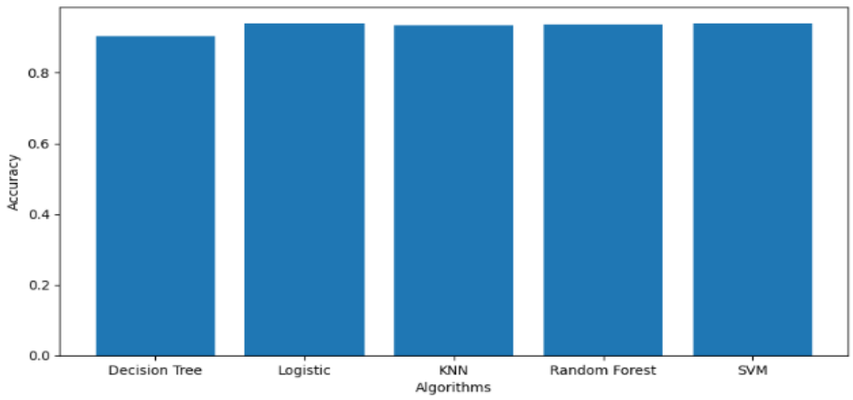
- ***Model Evaluation*:**

This part contains the explanation of how experiments are been carried out and how findings are obtained throughout the research. This project was carried out with the aid of the python programming language, and machine learning and deep learning techniques were used to carry out this research. To start this project, I have used csv data, loaded the dataset in CSV format by using the methods in python and applied preprocess technique and applied all the machine learning classifier models and predicted the performance metrics for that dataset. The train, test split function is divided such that the train split contains 80% and test split contains 20% of data from the dataset. For the csv file I have used various algorithms in the machine learning classification such as the Decision tree , Knearest Neighbor , Logistic Regression, SVM and Random Forest algorithm.

Accuracy score generated by each of the five models:

Model Used	*Accuracy Score*
Decision Tree	90.31
kNearest Neighbor	93.44
Logistic Regression	93.83
Random Forest	93.84
SVM	93.93

Bar graph representing the acccuracy score of each algorithm:



From the performance analysis of each algorithm we see that SVM , Random Forest and Logistic Regression has the top 3 best results with SVM achieving the highest accuracy slightly ahead of Random Forest. The model can help people with a cautionary indication of getting affected with stroke. The limitation of the steps was that the dataset used are not perfectly symmetrical. However it didn't effect the accuracy of the algorithms.

- ***Model selection:***

The Random Forest algorithm model was choosen as the base model to be integrated with the flask application. It returned 93.84% of accuracy and its test results indictaed that it accurately predicts and provides the result in the application.

The flask application was deployed into the localhost (port=5000). The homepage contains 10 features and input fields namely gender, age, hypertension, heart_disease, ever_married, work_type, Residence_type, avg_glucose_level, bmi, smoking_status and a button that is for to submit the input. On submission based on the input features and values the model predicts and informs whether the user is more likely to get a stroke or not

```
FINAL MODEL 'Random Forest Classifier
Accuracy Score value: 0.9384
              precision    recall  f1-score   support

           0       0.94      1.00      0.97       960
           1       0.00      0.00      0.00        62

    accuracy                           0.94      1022
   macro avg       0.47      0.50      0.48      1022
weighted avg       0.88      0.94      0.91      1022
```

6.2 CODING in jupyter:

1 **Importing Pandas, Numpy, Matplotlib, Seaborn and Warings Library.**

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
plt.rcParams['figure.figsize'] = (10, 5)
```

2 **Import the CSV Data as Pandas DataFrame**

data=pd.read_csv(r'C:\Users\Sneha\Documents\A\MSC 5th sem project\heart-stroke-prediction\heart-stroke-classifier\heart-stroke-classifier\Stroke-Risk-Prediction-using-Machine-Learning\dataset\healthcare-dataset-stroke-data.csv')

3 **Show Top 5 Records**

data

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1
...	...	...	...	...	...	...	...	...	...	...	...	...
5105	18234	Female	80.0	1	0	Yes	Private	Urban	83.75	NaN	never smoked	0
5106	44873	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.0	never smoked	0
5107	19723	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.6	never smoked	0
5108	37544	Male	51.0	0	0	Yes	Private	Rural	166.29	25.6	formerly smoked	0
5109	44679	Female	44.0	0	0	Yes	Govt_job	Urban	85.28	26.2	Unknown	0

Exploratory data analysis

data.shape

3 **Summary of the dataset**

data.describe()
data.info()
data.isnull().sum()

numeric_features = [feature for feature in data.columns if data[feature].dtype != 'O']
categorical_features = [feature for feature in data.columns if data[feature].dtype == 'O']

# print columns
print('We have {} numerical features : {}'.format(len(numeric_features), numeric_features))
print('\nWe have {} categorical features : {}'.format(len(categorical_features), categorical_features))

Feature Information

- Here we used the heart stroke dataset that is available in the kaggle website for our analysis. This dataset consists of total 12 attributes. The complete description of the attributes used in the proposed work is given below:
- **id** : This attribute means person's id. It's numerical data.
- **Age** : This attribute means a person's age. It's numerical data.
- **Gender** : This attribute means a person's gender. It's categorical data.
- **Hypertension** : This attribute means that this person is hypertensive or not. It's numerical data.
- **work type** : This attribute represents the person work scenario. It's categorical data.
- **residence type** : This attribute represents the person living scenario. It's

categorical data.

- **heart disease** : This attribute means whether this person has a heart disease person or not. It’s numerical data.
- **avg glucose level** : This attribute means what was the level of a person’s glucose condition. It’s numerical data.
- **Bmi** : This attribute means body mass index of a person. It’s numerical data.
- **ever married** : This attribute represents a person’s married status. It’s categorical data.
- **smoking Status** : This attribute means a person’s smoking condition. It’s categorical data.
- **Stroke** : This attribute means a person previously had a stroke or not.

```
for col in categorical_features:
    print(data[col].value_counts(normalize=True) * 100)
    print('--------------------------')
```

Univariate Analysis:
Numerical Features:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import math

# Sample DataFrame (replace with your actual data)
# df = pd.read_csv('your_data.csv')  # Load your dataset

# Identify numeric features
numeric_features = data.select_dtypes(include=[np.number]).columns.tolist()

# Dynamically calculate rows and columns for subplots
n_features = len(numeric_features)
n_cols = 2  # Number of columns
n_rows = math.ceil(n_features / n_cols)  # Dynamically calculate rows based on the number of features

plt.figure(figsize=(15, 5 * n_rows))  # Adjust height based on the number of rows

for i in range(n_features):
    plt.subplot(n_rows, n_cols, i + 1)

    # Clean the data by dropping NaN and infinite values
    data_to_plot = data[numeric_features[i]].dropna()
    data_to_plot = data_to_plot[~data_to_plot.isin([np.inf, -np.inf])]

    # Check if there's enough data to plot
    if len(data_to_plot) > 0:
        plt.hist(data_to_plot, bins=30, color='blue', alpha=0.7)
        plt.xlabel(numeric_features[i])
        plt.ylabel('Frequency')
        plt.title(f'Distribution of {numeric_features[i]}')
```

```
    else:
        plt.text(0.5, 0.5, 'No valid data', horizontalalignment='center',
verticalalignment='center')

# Automatically adjust subplot parameters for a neat layout
plt.tight_layout()
plt.show()
```

Categorial Features:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Assuming 'categorical_features' contains the column names for categorical data
# Check if categorical_features is not empty
if len(categorical_features) == 0:
    print("No categorical features to plot!")
else:
    n_features = len(categorical_features)
    n_cols = 3  # Number of columns in the subplot grid
    n_rows = (n_features + n_cols - 1) // n_cols  # Calculate rows needed

    plt.figure(figsize=(15, 8))
    plt.suptitle('Univariate Analysis of Categorical Features', fontsize=20,
fontweight='bold', alpha=0.8, y=1.05)

    for i in range(n_features):
        plt.subplot(n_rows, n_cols, i + 1)
        sns.countplot(x=data[categorical_features[i]], palette='viridis')
        plt.xlabel(categorical_features[i])
        plt.ylabel('Count')

    plt.tight_layout()
    plt.show()
```

Multivariate Anaalysis:

```
discrete_features=[feature for feature in numeric_features if
(len(data[feature].unique())<=25 and len(data[feature].unique())>5)]

continuous_features=[feature for feature in numeric_features if
len(data[feature].unique()) > 25]

encoded_categorical = [feature for feature in numeric_features if
len(data[feature].unique()) <=5]

print('We have {} discrete features : {}'.format(len(discrete_features),
discrete_features))
print('\nWe have {} continuous_features : {}'.format(len(continuous_features),
continuous_features))
print('\nWe have {} encoded_categorical : {}'.format(len(encoded_categorical),
encoded_categorical))
```

```
categorical_features = categorical_features + encoded_categorical
print(categorical_features)

data[(list(data[continuous_features])[1:])].corr()

plt.figure(figsize = (15,10))
cont_features = continuous_features.copy()
sns.heatmap(data[cont_features].corr(), cmap="CMRmap_r", annot=True)
plt.show()

from scipy.stats import chi2_contingency
chi2_test = []
for feature in categorical_features:
    if chi2_contingency(pd.crosstab(data['stroke'], data[feature]))[1] < 0.05:
        chi2_test.append('Reject Null Hypothesis')
    else:
        chi2_test.append('Fail to Reject Null Hypothesis')
result = pd.DataFrame(data=[categorical_features, chi2_test]).T
result.columns = ['Column', 'Hypothesis Result']
result

df1 = data.copy()
df1.gender = np.where(data.gender == 'Other', 'Female', data.gender)
chi2_contingency(pd.crosstab(df1['stroke'], df1['gender']))
```

Fill Null Values:

```
data['bmi'].value_counts()
data['bmi'].describe()
data['bmi'].fillna(data['bmi'].mean(),inplace=True)
data['bmi'].describe()
data.isnull().sum()
clr1 = ['#1E90FF', '#DC143C']
fig, ax = plt.subplots(4, 2, figsize=(12,20))
fig.suptitle('Distribution of Numerical Features By stroke', color='#3C3744',
             fontsize=20, fontweight='bold', ha='center')
for i, col in enumerate(continuous_features):
    sns.boxplot(data=data, x='stroke', y=col,  palette=clr1, ax=ax[i,0])
    ax[i,0].set_title(f'Boxplot of {col}', fontsize=12)
    sns.histplot(data=data, x=col, hue='stroke', bins=20, kde=True,
                 multiple='stack', palette=clr1, ax=ax[i,1])
    ax[i,1].set_title(f'Histogram of {col}', fontsize=14)
fig.tight_layout()
fig.subplots_adjust(top=0.90)
```

Visualitaion:

```
df1 = data.copy()
df1['stroke'] = np.where((data.stroke == 1),'Stroke', 'Not-Stroke' )
percentage = df1.stroke.value_counts(normalize=True)*100
label = ["Not-Stroke", "Stroke"]

# Plot PieChart with Ploty library
fig, ax = plt.subplots(figsize =(15, 8))
```

```
explode = (0, 0.1)
colors = ['#1188ff', '#e63a2a']
ax.pie(percentage, labels = label, startangle = 90,
       autopct='%1.2f%%',explode=explode, shadow=True, colors=colors)
plt.show()
```

Do Men or Women have more chance of stroke?

```
df1.columns
df1[df1.stroke == 'Stroke'].gender.value_counts(normalize=True)

plt.subplots(figsize=(14,7))
sns.countplot(x="gender",hue="stroke", data=df1,ec = "black",palette="Set2")
plt.title("gender vs stroke", weight="bold",fontsize=20, pad=20)
plt.ylabel("Package Count", weight="bold", fontsize=20)
plt.xlabel("Gender", weight="bold", fontsize=16)
plt.show()
```

AGE Category Split

```
bins= [0,10,20,50,85]
labels = ['Children','Teens','Adult','Senior']
df2 = data.copy()
df2['age_cat'] = pd.cut(data['age'], bins=bins, labels=labels, right=False)
age_group = df2.groupby(['age_cat',
'stroke'])['id'].count().reset_index(name='count')
age_group

plt.subplots(figsize=(14,7))
sns.countplot(x="age_cat",hue="stroke", data=df2,ec = "black",palette="Set1")
plt.title("Age Category vs stroke", weight="bold",fontsize=20, pad=20)
plt.ylabel("Count", weight="bold", fontsize=20)
plt.xlabel("Age Category", weight="bold", fontsize=16)
plt.show()

df2= data.copy()
bins=[0,10,20,30,40,50,60,70,80,90]
labels=['0-10','10-20','20-30','30-40','40-50','50-60','60-70','70-80','80-90']
df2['age_group']=pd.cut(data['age'],bins=bins,labels=labels)

import matplotlib.ticker as mtick

plt.figure(figsize=[14,7])

(100*df2[df2["stroke"].isin([1])]['age_group'].value_counts()/df2['age_group'].valu
e_counts()).plot(
   kind='bar',stacked=True , colormap='Accent')
plt.title("Percentage of stroke patients" , fontsize = 15, fontweight ='bold'  )
order1 =
(100*df2[df2["stroke"].isin([1])]['age_group'].value_counts()/df2['age_group'].valu
e_counts())
for n in range(order1.shape[0]):
   count = order1[n]
```

```
    strt='{:0.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('Decade' , fontweight ='bold')
plt.xticks(rotation=0)
plt.show()
```

Does HyperTension makes any difference:

```
plt.subplots(figsize=(14,7))
sns.countplot(x="hypertension", data=df2,ec = "black",palette="Set1",
hue='stroke')
plt.title("HyperTension vs Heart Stroke", weight="bold",fontsize=20, pad=20)
plt.ylabel("Count", weight="bold", fontsize=20)
plt.xlabel("Hypertension", weight="bold", fontsize=16)
plt.show()

plt.subplots(figsize=(14,7))

(100*df2[df2["stroke"].isin([1])]['hypertension'].value_counts()/df2['hypertension']
.value_counts()).plot(
    kind='bar',stacked=True , colormap='Accent')
plt.title("Percentage of stroke patients" , fontsize = 15, fontweight ='bold'  )
order1 =
(100*df2[df2["stroke"].isin([1])]['hypertension'].value_counts()/df2['hypertension']
.value_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:0.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('Hypertension' , fontweight ='bold')
plt.xticks(ticks=[0,1], labels=['No', 'Yes'] ,rotation=0)
plt.show()
```

Impact of BMI

```
fig1, ax1 = plt.subplots(figsize=(12,8))
ax1.pie(x=[data[data['bmi'] <= 18.5]['stroke'].value_counts()[0] ,
data[(data['bmi'] <= 25) & (data['bmi'] > 18)]['stroke'].value_counts()[0] ,
data[(data['bmi'] <= 30) & (data['bmi'] > 25)]['stroke'].value_counts()[0] ,
data[data['bmi'] > 30]['stroke'].value_counts()[0] ],
labels=['Under_weight','Normal_weight','Over_weight','Obese'] ,
pctdistance=0.6 , radius=6  , autopct='%1.1f%%' ,
colors=['powderblue','lavender','tan','pink'] )

ax1.axis('equal')
plt.legend(title = "Percentage of all patients for BMI category" , loc=1 ,
bbox_to_anchor=(1.2, 0.4))
plt.show()

df2= data.copy()
bins=[0,18,25,30,100]
```

```
labels=['0-18','18-25','25-30','30-100']
df2['bmi_cat']=pd.cut(data['bmi'],bins=bins,labels=labels)
bmi_group = df2.groupby(['bmi_cat',
'stroke'])['id'].count().reset_index(name='count')
bmi_group

plt.subplots(figsize=(14,7))
sns.countplot(x="bmi_cat",hue="stroke", data=df2,ec = "black",palette="Set1")
plt.title("BMI category vs stroke", weight="bold",fontsize=20, pad=20)
plt.ylabel("Count", weight="bold", fontsize=20)
plt.xlabel("BMI category", weight="bold", fontsize=16)
plt.show()

plt.subplots(figsize=(14,7))

(100*df2[df2["stroke"].isin([1])]['bmi_cat'].value_counts()/df2['bmi_cat'].value_co
unts()).plot(
    kind='bar',stacked=True , colormap='Accent')
plt.title("Percentage of stroke patients" , fontsize = 15, fontweight ='bold'  )
order1 =
(100*df2[df2["stroke"].isin([1])]['bmi_cat'].value_counts()/df2['bmi_cat'].value_co
unts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:0.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('BMI' , fontweight ='bold')
plt.xticks(ticks=[0,1,2,3],
labels=['Under_weight','Normal_weight','Over_weight','Obese'][::-1] ,rotation=0)
plt.show()
```

Heark disease vs Stroke

```
plt.subplots(figsize=(14,7))
sns.countplot(x="heart_disease",hue='stroke', data=data,ec =
"black",palette="Set3")
plt.title("Heart disease vs stroke", weight="bold",fontsize=20, pad=20)
plt.ylabel("Heart disease", weight="bold", fontsize=20)
plt.xlabel("count", weight="bold", fontsize=16)
plt.show()

plt.subplots(figsize=(14,7))

(100*df2[df2["stroke"].isin([1])]['heart_disease'].value_counts()/df2['heart_disease'
].value_counts()).plot(
    kind='bar',stacked=True , colormap='Accent')
plt.title("Percentage of stroke patients" , fontsize = 15, fontweight ='bold'  )
order1 =
(100*df2[df2["stroke"].isin([1])]['heart_disease'].value_counts()/df2['heart_disease'
].value_counts())
for n in range(order1.shape[0]):
    count = order1[n]
```

```
    strt='{:0.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('Heart disease' , fontweight ='bold')
plt.xticks(ticks=[0,1], labels=['No', 'Yes'] ,rotation=0)
plt.show()
```

Does marriage have any effect on the stroke

```
plt.subplots(figsize=(14,7))
sns.countplot(x="ever_married",hue='stroke', data=data,ec =
"black",palette="Set3")
plt.title("Married vs stroke", weight="bold",fontsize=20, pad=20)
plt.xlabel("Married", weight="bold", fontsize=20)
plt.ylabel("count", weight="bold", fontsize=16)
plt.show()

plt.subplots(figsize=(14,7))

(100*df2[df2["stroke"].isin([1])]['ever_married'].value_counts()/df2['ever_married'
].value_counts()).plot(
    kind='bar',stacked=True , colormap='Accent')
plt.title("Percentage of stroke patients" , fontsize = 15, fontweight ='bold'  )
order1 =
(100*df2[df2["stroke"].isin([1])]['ever_married'].value_counts()/df2['ever_married'
].value_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:0.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('Married' , fontweight ='bold')
plt.xticks(ticks=[0,1], labels=['Yes', 'No'] ,rotation=0)
plt.show()
```

Work type vs Stroke

```
plt.subplots(figsize=(14,7))
sns.countplot(x="work_type",hue='stroke', data=data,ec = "black",palette="Set3")
plt.title("Work type vs stroke", weight="bold",fontsize=20, pad=20)
plt.xlabel("Work type", weight="bold", fontsize=20)
plt.ylabel("count", weight="bold", fontsize=16)
plt.show()

df2 = data.copy()
df2['work_type'] = np.where(df2['work_type'] == 'children', 'Never_worked',
df2['work_type'])
plt.subplots(figsize=(14,7))
sns.countplot(x="work_type",hue='stroke', data=df2,ec = "black",palette="Set3")
plt.title("Work type vs stroke", weight="bold",fontsize=20, pad=20)
plt.ylabel("Work type", weight="bold", fontsize=20)
plt.xlabel("count", weight="bold", fontsize=16)
plt.show()
```

```
plt.subplots(figsize=(14,7))

(100*df2[df2["stroke"].isin([1])]['work_type'].value_counts()/df2['work_type'].val
ue_counts()).plot(
    kind='bar',stacked=True , colormap='Accent' )
plt.title("Percentage of stroke patients" , fontsize = 15, fontweight ='bold'  )
order1 =
(100*df2[df2["stroke"].isin([1])]['work_type'].value_counts()/df2['work_type'].val
ue_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:0.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('Type of jobs' , fontweight ='bold')
plt.xticks(rotation=0)
plt.show()
```

Resident type vs stroke

```
plt.subplots(figsize=(14,7))
sns.countplot(x="Residence_type",hue='stroke', data=df2,ec =
"black",palette="Set3")
plt.title("Resident type vs stroke", weight="bold",fontsize=20, pad=20)
plt.ylabel("Resident type", weight="bold", fontsize=20)
plt.xlabel("count", weight="bold", fontsize=16)
plt.show()

plt.subplots(figsize=(14,7))

(100*df2[df2["stroke"].isin([1])]['Residence_type'].value_counts()/df2['Residence_
type'].value_counts()).plot(kind='bar',stacked=True , colormap='Accent' )
plt.title("Percentage of stroke patients" , fontsize = 15, fontweight ='bold'  )
order1 =
(100*df2[df2["stroke"].isin([1])]['Residence_type'].value_counts()/df2['Residence_
type'].value_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:0.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('Resident type' , fontweight ='bold')
plt.xticks(rotation=0)
plt.show()
```

Effect of glucose level

```
sns.histplot(data.avg_glucose_level,kde=True)

data.avg_glucose_level.min(), data.avg_glucose_level.max()

bins= [0,70,140,200, 300]
labels = ['low','Normal','High','Diabetic']
```

```
df2 = data.copy()
df2['glucose_cat'] = pd.cut(data['avg_glucose_level'], bins=bins, labels=labels,
right=False)
age_group = df2.groupby(['glucose_cat',
'stroke'])['id'].count().reset_index(name='count')
age_group

plt.subplots(figsize=(14,7))
sns.countplot(x="glucose_cat",hue="stroke", data=df2,ec = "black",palette="Set1")
plt.title("Glucose category vs stroke", weight="bold",fontsize=20, pad=20)
plt.ylabel("Count", weight="bold", fontsize=20)
plt.xlabel("Glucose category", weight="bold", fontsize=16)
plt.show()

plt.subplots(figsize=(14,7))

(100*df2[df2["stroke"].isin([1])]['glucose_cat'].value_counts()/df2['glucose_cat'].v
alue_counts()).plot(kind='bar',stacked=True , colormap='Accent' )
plt.title("Percentage of stroke patients" , fontsize = 15, fontweight ='bold'  )
order1 =
(100*df2[df2["stroke"].isin([1])]['glucose_cat'].value_counts()/df2['glucose_cat'].v
alue_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:0.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('Glucose level' , fontweight ='bold')
plt.xticks(ticks=[0,1,2,3], labels = ['low','Normal','High','Diabetic'] ,rotation=0)
plt.show()
```

smoking status vs stroke:

```
plt.subplots(figsize=(14,7))
sns.countplot(x="smoking_status",hue='stroke', data=df2,ec =
"black",palette="Set3")
plt.title("Smoking Status vs stroke", weight="bold",fontsize=20, pad=20)
plt.xlabel("Smoking Status", weight="bold", fontsize=20)
plt.ylabel("count", weight="bold", fontsize=16)
plt.show()

plt.subplots(figsize=(14,7))
(100*df2[df2["stroke"].isin([1])]['smoking_status'].value_counts()/df2['smoking_st
atus'].value_counts()).plot(kind='bar',stacked=True , colormap='Accent' )
plt.title("Percentage of stroke patients" , fontsize = 15, fontweight ='bold'  )
order1 =
(100*df2[df2["stroke"].isin([1])]['smoking_status'].value_counts()/df2['smoking_st
atus'].value_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:0.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')
```

```
plt.xlabel('Smoking status' , fontweight ='bold')
plt.xticks(rotation=0)
plt.show()

column_list = ['age', 'avg_glucose_level', 'bmi']

fig, ax = plt.subplots(1, 3, figsize=(12,6))

for i, col in enumerate(column_list):
  sns.boxplot(data=data, x='stroke', y=col, ax=ax[i])

plt.tight_layout()
plt.show()
```

Data Pre-Procressing:

```
data.drop('id',axis=1,inplace=True)
data.duplicated().sum()
data
from matplotlib.pyplot import figure
figure(num=None, figsize=(8, 6), dpi=800, facecolor='w', edgecolor='k')

data.plot(kind='box')
plt.show()
```

Feature Engeneering:

```
numeric_features = [feature for feature in data.columns if data[feature].dtype !=
'O']
print('Num of Numerical Features :', len(numeric_features))
numeric_features

categorical_features = [feature for feature in data.columns if data[feature].dtype ==
'O']
print('Num of Categorical Features :', len(categorical_features))
categorical_features

discrete_features=[feature for feature in numeric_features if
(len(data[feature].unique())<=25)]
print('Num of Discrete Features :',len(discrete_features))
discrete_features

continuous_features=[feature for feature in numeric_features if
len(data[feature].unique()) > 25]
print('Num of Continuous Features :',len(continuous_features))
continuous_features

data[continuous_features].skew(axis=0)

plt.figure(figsize=(12, 6))
for i, col in enumerate(continuous_features):
   plt.subplot(2, 2, i+1)
```

```
    sns.kdeplot(x=df1[col], color='indianred')
    plt.xlabel(col)
    plt.tight_layout()
```

Label Encoding:

```
data.head()
pip install scipy
from sklearn.preprocessing import LabelEncoder
enc=LabelEncoder()

gender=enc.fit_transform(data['gender'])
moking_status=enc.fit_transform(data['smoking_status'])

work_type=enc.fit_transform(data['work_type'])
Residence_type=enc.fit_transform(data['Residence_type'])
ever_married=enc.fit_transform(data['ever_married'])

data['work_type']=work_type

data['ever_married']=ever_married
data['Residence_type']=Residence_type
data['smoking_status']=smoking_status
data['gender']=gender

data

data.info()
```

Partioning:

```
X ---train_X,test_X  80/20
Y ---train_Y,test_Y

X=data.drop('stroke',axis=1)

X.head()
Y=data['stroke']

Y
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train,
Y_test=train_test_split(X,Y,test_size=0.2,random_state=10)

X_train

Y_train

X_test

Y_test

data.describe()
```

```
from sklearn.preprocessing import StandardScaler
std=StandardScaler()

X_train_std=std.fit_transform(X_train)
X_test_std=std.transform(X_test)\
```

Save the scalar object:

```
import pickle
import os

scaler_path=os.path.join('C:/Users/Sneha/Documents/A/MSC 5th sem
project/heart-stroke-prediction/heart-stroke-classifier\heart-stroke-
classifier/Stroke-Risk-Prediction-using-Machine-Learning/models/scaler.pkl')
with open(scaler_path,'wb') as scaler_file:
    pickle.dump(std,scaler_file)

X_train_std

X_test_std
```

Training Models:

Decision Tree:

```
from sklearn.tree import DecisionTreeClassifier
dt=DecisionTreeClassifier()
dt.fit(X_train_std,Y_train)
dt.feature_importances_
X_train.columns
Y_pred=dt.predict(X_test_std)
Y_pred
from sklearn.metrics import accuracy_score
ac_dt=accuracy_score(Y_test,Y_pred)
ac_dt

import joblib
model_path=os.path.join('C:/Users/Sneha/Documents/A/MSC 5th sem
project/heart-stroke-prediction/heart-stroke-classifier/heart-stroke-
classifier/Stroke-Risk-Prediction-using-Machine-Learning/models/dt.sav')
joblib.dump(dt,model_path)
```

Logistic Regression:

```
from sklearn.linear_model import LogisticRegression
lr=LogisticRegression()
lr.fit(X_train_std,Y_train)
Y_pred_lr=lr.predict(X_test_std)
Y_pred_lr=lr.predict(X_test_std)
Y_pred_lr
ac_lr=accuracy_score(Y_test,Y_pred_lr)
ac_lr
```

```
import joblib
model_path=os.path.join('C:/Users/Sneha/Documents/A/MSC 5th sem
project/heart-stroke-prediction/heart-stroke-classifier/heart-stroke-
classifier/Stroke-Risk-Prediction-using-Machine-Learning/models/lr.sav')
joblib.dump(lr,model_path)
```

Knn

```
from sklearn.neighbors import KNeighborsClassifier
knn=KNeighborsClassifier()
knn.fit(X_train_std,Y_train)
Y_pred=knn.predict(X_test_std)
ac_knn=accuracy_score(Y_test,Y_pred)
ac_knn
```

```
import joblib
model_path=os.path.join('C:/Users/Sneha/Documents/A/MSC 5th sem
project/heart-stroke-prediction/heart-stroke-classifier/heart-stroke-
classifier/Stroke-Risk-Prediction-using-Machine-Learning/models/knn.sav')
joblib.dump(knn,model_path)
```

Random Forest:

```
from sklearn.ensemble import RandomForestClassifier
rf=RandomForestClassifier()
rf.fit(X_train_std,Y_train)
Y_pred=rf.predict(X_test_std)
ac_rf=accuracy_score(Y_test,Y_pred)
ac_rf
```

```
import joblib
model_path=os.path.join('C:/Users/Sneha/Documents/A/MSC 5th sem
project/heart-stroke-prediction/heart-stroke-classifier/heart-stroke-
classifier/Stroke-Risk-Prediction-using-Machine-Learning/models/rf.sav')
joblib.dump(rf,model_path)
```

SVM:

```
from sklearn.svm import SVC
sv=SVC()
sv.fit(X_train_std,Y_train)
Y_pred=sv.predict(X_test_std)
ac_sv=accuracy_score(Y_test,Y_pred)
ac_sv
```

```
import joblib
model_path=os.path.join('C:/Users/Sneha/Documents/A/MSC 5th sem
project/heart-stroke-prediction/heart-stroke-classifier/heart-stroke-
classifier/Stroke-Risk-Prediction-using-Machine-Learning/models/sv.sav')
joblib.dump(sv,model_path)
```

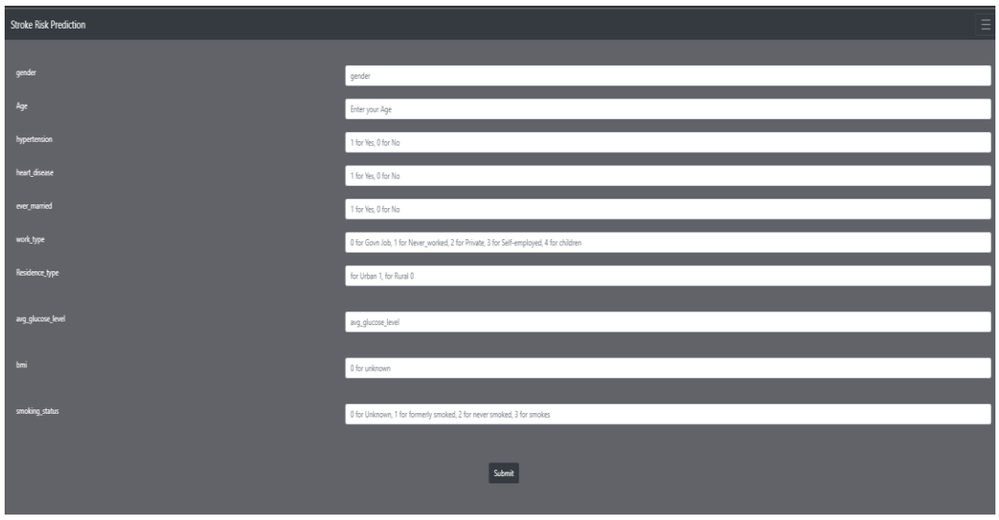
```
plt.bar(['Decision Tree','Logistic','KNN','Random
Forest','SVM'],[ac_dt,ac_lr,ac_knn,ac_rf,ac_sv])
```

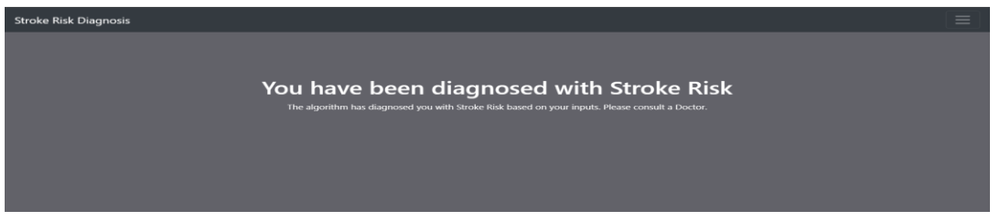
```
plt.xlabel("Algorithms")
plt.ylabel("Accuracy")
plt.show()
```

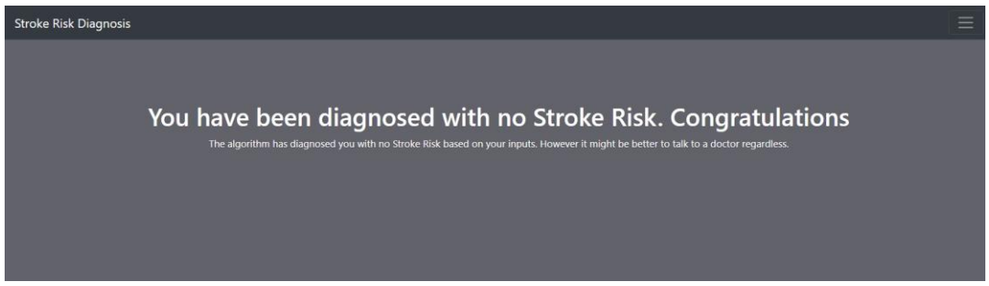
Chossing Best Model:

```
from sklearn.metrics import accuracy_score, classification_report

best_model = RandomForestClassifier()
best_model = best_model.fit(X_train_std, Y_train)

# Predict on test data
Y_pred = best_model.predict(X_test_std)

# Calculate accuracy score
score = accuracy_score(Y_test, Y_pred)

cr = classification_report(Y_test, Y_pred)

print("FINAL MODEL 'Random Forest Classifier")
print("Accuracy Score value: {:.4f}".format(score))
print(cr)
```

Output Screenshots:







Chapter 7

CONCLUSION

Through this project a sufficently large dataset of stroke attacked patients has been successfully classified. Random Forest, Decision Tree, KNN, SVM and Logistic Regression were used for training and testing the model to detect stroke in patients. From the performance analysis we see that Random forest performed well to detect and proivde accurate results. The novelty and the main contribution of my project is collecting this dataset and preparing them to use with the application. The model can help people with a cautionary indication of being getting effected by stroke disease

Chapter 8

FUTURE SCOPE

In future work, it is possible to extend the research by using different classification techniques and models to predict stroke disease more accurately by modiying this proposed model and improving its performance. Moreover to create a fully functioning hospital management system with login credentials for various users and various features pages will integrate this stroke detection feature within that application.

Chapter 9

REFERENCES

[1] D. Zhong, Y. Chen, Y. Chen, S. Ye, W. Cai, and M. Chen, “A Classification Method Based on Deep Convolutional Neural Network,” Journal of Healthcare Engineering, vol. 2021, 2021, doi: 10.1155/2021/7167891.

[2] T. M. Geethanjali, D. M. D, M. S. K, S. M. K, and A. Professor, “STROKE PREDICTION USING MACHINE LEARNING,” JETIR, 2021. [Online]. Available: www.jetir.orgd710

[3] B. Deepak Kumar, S. Yellaram, S. kothamasu, S. Puchakayala, and A. Professor, “Heart Stroke Prediction using Machine Learning,” 2021. [Online]. Available: www.ijcrt.org

[4] G. Sailasya and G. L. Aruna Kumari, “Analyzing the Performance of Stroke Prediction using ML Classification Algorithms.” [Online]. Available: www.ijacsa.thesai.org

[5] L. Amini, R. Azarpazhouh, M. T. Farzadfar, S. A. Mousavi, F. Jazaieri, F. Khorvash, R. Norouzi, and N. Toghianfar, “Prediction and control of heart stroke by data mining,” International Journal of Preventive Medicine, vol. 4, no. Suppl 2, pp. S245–249, May 2013.

[6] P. Govindarajan, R. K. Soundarapandian, A. H. Gandomi, R. Patan, P. Jayaraman, and R. Manikandan, “Classification of heart stroke disease using machine learning algorithms,” Neural Computing and Applications, vol. 32, no. 3, pp. 817–828, Feb. 2020.